MISSAO: Fabrica Inteligente TechInova

A TechInova e uma indústria de manufatura que esta migrando para o modelo de Industria 4.0. Você foi contratado como DevOps Engineer júnior e sua primeira missão e containerizar o micro serviço de monitoramento de sensores IoT da linha de produção — garantindo que ele rode de forma idêntica em qualquer ambiente, do notebook do desenvolvedor ate o servidor de produção na nuvem.

Ao concluir este lab, você terá criado sua primeira imagem Docker pronta para ser implantada em um cluster Kubernetes na AWS.

Industria 4.0: Por que containers importam?

Na Industria 4.0, sensores, robos e sistemas de controle geram dados em tempo real. Cada micro serviço deve ser:

Isolado — falha em um não derruba os outros
Portátil — roda no edge computing da fábrica ou na nuvem
Escalável — instancias sobem e descem conforme a demanda de dados

Containers Docker sao a base tecnológica que viabiliza essa arquitetura.

Objetivos de Aprendizagem

Ao final deste laboratório você será capaz de:

- Criar a estrutura de um projeto Docker para uma aplicação Python/Flask
- Escrever um Dockerfile com as instruções essenciais (FROM, WORKDIR, COPY, RUN, EXPOSE, CMD)
- Construir uma imagem Docker e instanciar containers localmente
- Entender como esse fluxo se aplica a soluções reais de Industria 4.0

Parte 1 — Construindo o Microservico de Monitoramento IoT

Neste exercício você vai criar e containerizar uma API Flask que simula o endpoint de coleta de dados de sensores industriais — exatamente o tipo de micro serviço usado em plataformas de IIoT (Industrial Internet of Things).

1 Crie a estrutura do projeto

Crie um novo diretório para o projeto. Ele representara o repositório do micro serviço de monitoramento:

```
mkdir iot-monitor && cd iot-monitor
```

Estrutura que você irá criar ao longo do exercício:

```
código
iot-monitor/
    app.py           <- API Flask (microservico de sensores)
    requirements.txt <- dependencias Python
    Dockerfile       <- instrucoes para criar a imagem Docker
```

2 Crie o arquivo app.py — a API do sensor IoT

Crie um arquivo chamado app.py. Este micro serviço simula o endpoint de um sensor industrial que retorna dados de temperatura e status da linha de produção:

```
código
from flask import Flask, jsonify

app = Flask(__name__)

# Simulacao de dados de sensor IoT de uma linha de producao
@app.route('/')
def status():
    return 'SENAI Tech — DevOps | Aluno: <Coloque seu nome>'

@app.route('/sensor')
def sensor_data():
    return jsonify({
        'linha': 'Linha-A1',
        'sensor': 'Temperatura',
        'valor': 72.4,
        'unidade': 'Celsius',
        'status': 'NORMAL',
```

```
        'aluno': '<Coloque seu nome>'
    })

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0')
```

Contexto Industrial

Em uma fábrica real, este endpoint seria consumido por um sistema SCADA ou plataforma de IIoT (como AWS IoT Core) para monitorar equipamentos em tempo real, disparar alertas e alimentar dashboards de produção.

3 Crie o arquivo requirements.txt

O requirements.txt lista as dependências Python do micro serviço. Crie o arquivo com o seguinte conteúdo:

codigo

```
Flask
```

4 Escreva o Dockerfile

O Dockerfile e o 'manual de montagem' da sua imagem. Crie o arquivo Dockerfile (sem extensão) com as instruções abaixo:

código

```
# Imagem base: Python 3.9 slim (leve, ideal para microsservicos IoT)
FROM python:3.9-slim

# Diretorio de trabalho dentro do container
WORKDIR /app

# Copia os arquivos para o container
COPY app.py .
COPY requirements.txt .

# Instala as dependencias
RUN pip install -r requirements.txt
```

```
# Porta 5000: endpoint da API do sensor
EXPOSE 5000

# Inicia o microservico ao subir o container
CMD ["python", "app.py"]
```

Dicionário de Instruções do Dockerfile

Instrucao	O que faz
`FROM`	Define a imagem base. Tudo começa daqui — como escolher o sistema operacional de um servidor.
`WORKDIR`	Define o diretório padrão dentro do container. Evita erros de caminho em instruções seguintes.
`COPY`	Transfere arquivos do seu computador para dentro do container durante o build.
`RUN`	Executa comandos durante o build (instalação de pacotes, compilação, etc.).
`EXPOSE`	Documenta qual porta a aplicação usa. Não abre a porta — isso e feito no docker run.
`CMD`	Comando executado quando o container inicia. Ponto de entrada do seu micro serviço.

5 Construa a imagem Docker

Com o terminal aberto na pasta iot-monitor, execute o comando de build. O . indica que o Dockerfile está no diretório atual:

```
docker build . -t iot-monitor:1.0
```

Acompanhe a saída do terminal. Cada linha representa uma instrução do Dockerfile sendo executada. O Docker faz cache de cada camada — rebuilds futuros serao muito mais rapidos.

6 Execute o container e teste a API

Inicie o container mapeando a porta 5000 do container para a porta 5000 do seu computador:

```
docker run -p 5000:5000 iot-monitor:1.0
```

Abra o navegador e acesse os endpoints:

- http://localhost:5000 → pagina principal com seu nome
- http://localhost:5000/sensor → JSON com dados do sensor IoT

Você também pode visualizar o container em execução no Docker Desktop ou via terminal:

```
docker ps
```

Desafio Extra — Nivel DevOps

Você terminou antes do tempo? Tente estas extensões:

Adicione um novo endpoint /health que retorne {'status': 'ok'}
Adicione uma variavel de ambiente LINE_NAME no Dockerfile usando ENV
e use-a no codigo: os.environ.get('LINE_NAME', 'Linha-A1')
Construa uma versao 2.0 da imagem: docker build . -t iot-monitor:2.0
Compare o tamanho das imagens: docker images | grep iot-monitor

Reflexao Final

Responda no seu caderno ou ferramenta de anotações:

1. Por que usar FROM python:3.9-slim em vez de FROM python:3.9 completo?
2. O que aconteceria se você esquecesse a instrução EXPOSE 5000?
3. Em uma fábrica com 50 sensores, como você escalaria esse micro serviço?

Bons estudos!

Prof. Nisflei Galoni